

The logo for FIAP (Faculdade Integrada de Administração e Planejamento) is located at the top center. It consists of the letters 'FIAP' in a bold, sans-serif font, followed by a square icon containing a stylized 'ON' and the letters 'FI' to its right.

FIAP ON

The background of the slide features a grayscale image of the Earth, centered on the Atlantic Ocean. Overlaid on the globe is a complex network of white lines representing global connectivity. Various white line-art icons are scattered across the globe, including a cargo ship, a truck, an airplane, a train, a factory, a server rack, a mail envelope, and a computer monitor. The overall theme is global logistics and technology.

09

O ENCONTRO DO IOT COM A NUVEM

LISTA DE FIGURAS

Figura 1 - Pilha de protocolos aplicáveis a sistemas IoT	8
Figura 2 - Comparativo de desempenho entre os protocolos MQTT e CoAP	9
Figura 3 - Representação da arquitetura MQTT baseada em publicação e assinatura	10
Figura 4 - Representação do papel do broker MQTT no ecossistema IoT, demonstrando a comunicação entre sensores, serviços de backend e dispositivos de acesso	13
Figura 5 - Tela inicial do MQTT Explorer com árvore de tópicos carregada após conexão com broker local	18
Figura 6 - Publicação manual de mensagens JSON no tópico sensor/distancia utilizando o MQTT Explorer	19
Figura 7 - Tópicos do sistema (\$SYS) mostrando estatísticas do broker, incluindo mensagens, clientes e conexões ativas	20
Figura 8 - Execução do comando mosquitto_sub aguardando mensagens no tópico sensor/distancia	21
Figura 9 - Comando mosquitto_pub enviado com sucesso para o tópico sensor/distancia	22
Figura 10 - Terminal exibindo a mensagem recebida, validando o funcionamento do broker Mosquitto local	23
Figura 11 - Diagrama geral do experimento de semáforo inteligente com ESP32 e MQTT	24
Figura 12 - Diagrama de ligação entre o sensor ultrassônico HC-SR04 e o ESP32	25
Figura 13 - Diagrama de ligação dos LEDs ao ESP32 simulando um semáforo	26
Figura 14 - Configuração da conexão com o broker público no MQTT Explorer	31
Figura 15 - Assinatura do tópico sensor/aluno no MQTT Explorer	32
Figura 16 - Mensagens recebidas no monitor serial do ESP32 indicando distâncias inferiores a 30 cm	33
Figura 17 - Mensagens recebidas no tópico sensor/aluno no MQTT Explorer	34

LISTA DE QUADRO

Quadro 1 - Comparação entre três brokers públicos MQTT comumente utilizados para fins educacionais e testes em projetos de IoT	14
--	----

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Trecho de configuração básica do broker Mosquitto	16
Código-fonte 2 - Código de leitura do sensor ultrassônico HC-SR04 com ESP32....	28
Código-fonte 3 - Exemplo de mensagem JSON formatada para envio pelo dispositivo ESP32 com sensor ultrassônico, representando o payload publicado via protocolo MQTT	28
Código-fonte 4 - Publicador MQTT com sensor ultrassônico via ESP32	31
Código-fonte 5 - Assinante MQTT com ESP32 e controle de semáforo via LED	38

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Comando para iniciar a escuta no tópico sensor/distancia através do terminal, utilizando o broker MQTT local na porta 1883	21
Comando de prompt 2 - Publicação manual de uma mensagem JSON com valor de distância igual a 35, simulando um dispositivo publicador em um tópico MQTT	22

SUMÁRIO

1 O ENCONTRO DO IOT COM A NUVEM	7
1.1 A importância da comunicação em tempo real na IoT	7
1.2 O que é o protocolo MQTT	8
1.3 Arquitetura publish/subscribe	9
2 BROKERS MQTT	12
2.1 O papel do broker na arquitetura MQTT	12
2.2 Utilização de brokers públicos	13
2.3 Instalação de um broker local com Mosquitto	14
2.3.1 Instalação em diferentes sistemas operacionais	14
2.3.2 Configuração básica do Mosquitto	15
2.3.3 Testes com MQTT Explorer ou comandos MQTT CLI	17
2.3.3.1 Utilizando o MQTT Explorer	17
2.3.3.2 Utilizando o Mosquitto CLI	20
3 EXPERIMENTOS COM MQTT E ESP32	24
3.1 Fundamentos do circuito: sensor ultrassônico e LEDs	24
3.2 Enviando dados com sensor ultrassônico (publicador)	28
3.2.1 Assinatura do Tópico	31
3.2.2 Envio e Recepção de Mensagens	32
3.3 Recebendo dados e atuando com semáforo inteligente (assinante)	34
4 APLICAÇÕES REAIS E REFLEXÕES	39
REFERÊNCIAS	41

1 O ENCONTRO DO IOT COM A NUVEM

1.1 A importância da comunicação em tempo real na IoT

A Internet das Coisas (IoT) tem transformado diversos setores ao conectar dispositivos físicos a sistemas digitais, promovendo automação, monitoramento e tomada de decisão de forma mais rápida e eficiente. Em contextos como o controle de tráfego, a automação industrial, a agricultura de precisão e os sistemas de segurança, a capacidade de trocar informações em tempo real entre sensores, atuadores e plataformas computacionais é fundamental.

Quando um sensor detecta uma mudança no ambiente, como a presença de uma pessoa ou o aumento da temperatura, é essencial que essa informação seja transmitida de forma imediata a outro sistema, que pode reagir acionando um alarme, acionando uma ventilação ou armazenando o dado em nuvem. Para garantir esse fluxo contínuo de informação, mesmo em dispositivos com recursos limitados e conectividade instável, são necessários protocolos de comunicação eficientes, confiáveis e leves. É nesse contexto que o MQTT se destaca.

A figura a seguir apresenta a pilha de protocolos utilizada em arquiteturas de IoT, evidenciando como diferentes tecnologias se organizam nas camadas de aplicação, transporte, rede e enlace físico. Essa representação permite compreender a integração de protocolos como MQTT, CoAP, TCP, IPv6, ZigBee, entre outros, dentro de um sistema IoT.

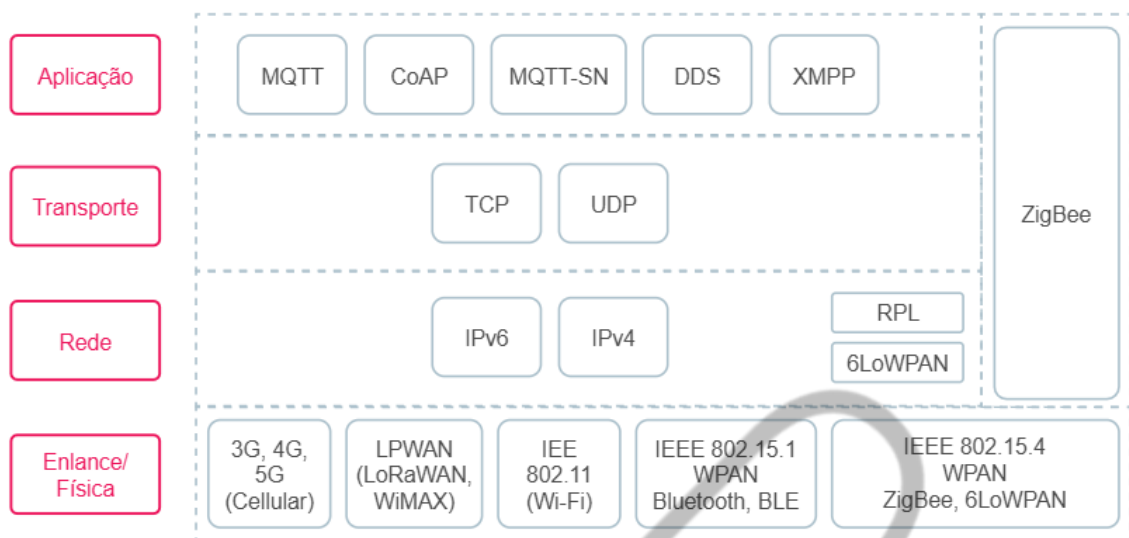


Figura 1 - Pilha de protocolos aplicáveis a sistemas IoT
Fonte: Faria Júnior (2025), adaptado por FIAP (2026)

1.2 O que é o protocolo MQTT

O MQTT, sigla para Message Queuing Telemetry Transport, é um protocolo leve de mensagens idealizado para ambientes com restrições de banda e dispositivos com capacidade limitada de processamento e memória. Ele surgiu com o objetivo de facilitar a comunicação entre equipamentos industriais e servidores, mas foi rapidamente adotado por aplicações de IoT devido à sua simplicidade e eficiência.

Funcionando sobre a pilha TCP/IP, o MQTT utiliza portas padrão como 1883 para conexões não seguras e 8883 para conexões criptografadas. Ele é compatível com diversas linguagens de programação e sistemas embarcados, o que o torna extremamente flexível para integrações com microcontroladores, aplicativos móveis e servidores em nuvem.

Entre seus pontos fortes estão o pequeno tamanho dos pacotes transmitidos, o suporte à comunicação bidirecional e a capacidade de operar mesmo em conexões intermitentes. O MQTT também permite controlar a entrega das mensagens por meio de diferentes níveis de qualidade de serviço, pode implementar autenticação e criptografia, e dispõe de mecanismos que toleram desconexões temporárias sem perda de dados críticos.

A próxima figura apresenta gráficos comparativos de desempenho entre os protocolos MQTT e CoAP, analisando o tempo de conexão e publicação com e sem

O Encontro do IoT com a Nuvem

autenticação. Esses resultados são fundamentais para avaliar a viabilidade de uso desses protocolos em dispositivos de IoT com restrições de energia e processamento.

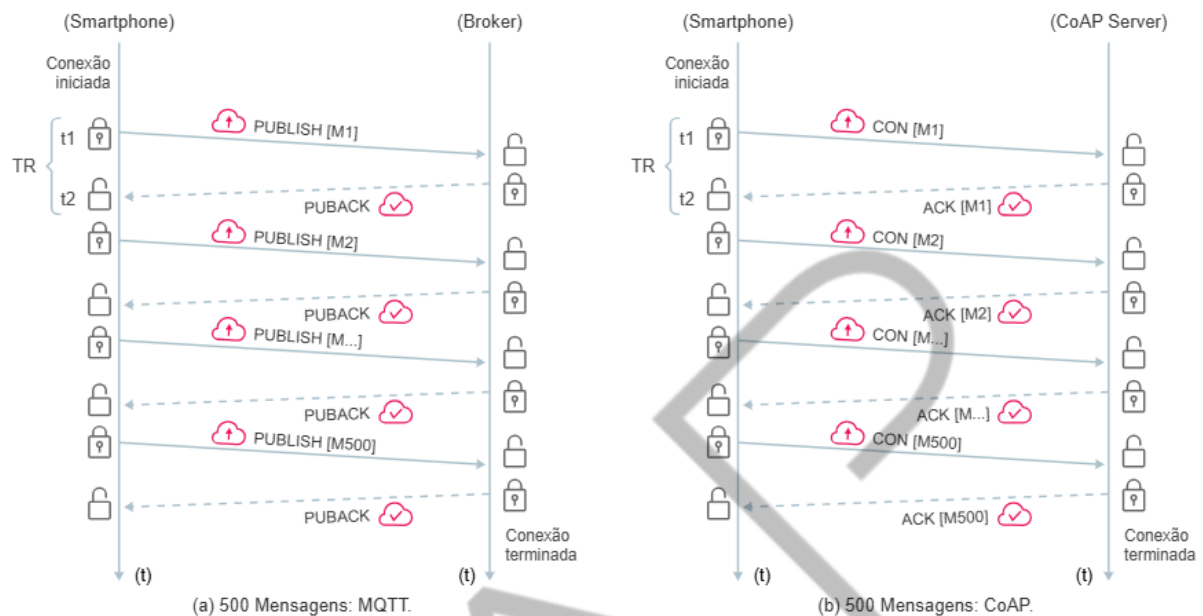


Figura 2 - Comparativo de desempenho entre os protocolos MQTT e CoAP
Fonte: ALFACOMP (2025), adaptado por FIAP (2026)

1.3 Arquitetura publish/subscribe

A arquitetura adotada pelo MQTT é baseada no modelo de publicação e assinatura, conhecido como publish/subscribe. Nesse modelo, os dispositivos que enviam mensagens são chamados de publicadores, enquanto aqueles que recebem são os assinantes. O tráfego entre eles é intermediado por um broker, que atua como um servidor central, recebendo as mensagens dos publicadores e repassando-as apenas para os assinantes interessados em um determinado tema.

Esse modelo é eficiente e escalável porque desacopla o remetente e o destinatário da mensagem. Ou seja, um sensor pode publicar um dado sem saber quem irá utilizá-lo, e diversos dispositivos podem receber essa mesma informação ao assinarem o tópico correspondente. Isso facilita a integração de novos elementos à rede e torna a comunicação mais dinâmica.

Os tópicos MQTT funcionam como categorias organizadas em uma estrutura hierárquica, como, por exemplo: "casa/sala/temperatura" ou "fabrica/setor1/pressao".

O Encontro do IoT com a Nuvem

Essa estrutura permite filtrar e agrupar mensagens de forma lógica, além de facilitar assinaturas globais com o uso de curingas.

O protocolo também oferece três níveis de qualidade de serviço (Quality of Service — QoS), que controlam a confiabilidade da entrega das mensagens. No nível 0, a mensagem é enviada apenas uma vez, sem confirmação. No nível 1, ela é garantida ao menos uma vez, com confirmação de recebimento. Já no nível 2, a entrega é garantida exatamente uma vez, utilizando uma troca mais complexa de confirmações entre o cliente e o broker.

A figura a seguir apresenta uma análise comparativa de desempenho entre três protocolos de mensagens: MQTT, AMQP e STOMP, aplicados em contextos IoT. Os gráficos foram organizados em três categorias distintas:

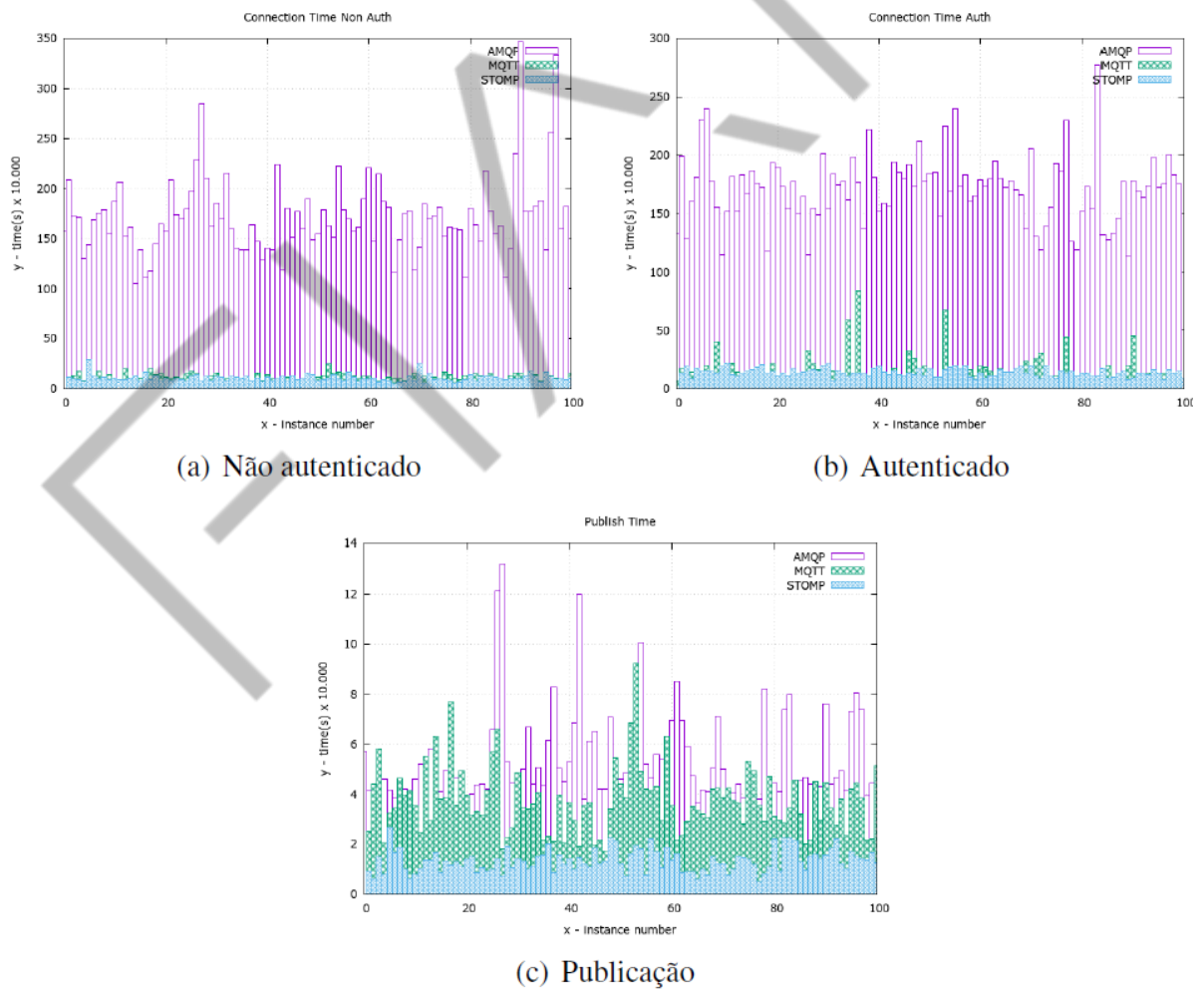


Figura 3 - Representação da arquitetura MQTT baseada em publicação e assinatura
Fonte: TEDESCHI (2025)

- **(a) Conexão sem autenticação:** mostra o tempo necessário para que cada protocolo estabeleça conexão com o broker sem exigir credenciais. Observa-se que o AMQP apresenta os maiores tempos médios, enquanto o MQTT mantém consistência e baixa variação, evidenciando sua leveza.
- **(b) Conexão com autenticação:** apresenta o desempenho com mecanismos de segurança ativados. Apesar do aumento de tempo em todos os protocolos, o MQTT continua apresentando os menores tempos de conexão, seguido de perto pelo STOMP.
- **(c) Tempo de publicação:** compara o tempo necessário para transmitir uma mensagem do cliente para o broker. Novamente, o MQTT se destaca pela sua eficiência, sendo mais rápido e mais estável em comparação aos demais, mesmo em cenários com múltiplas instâncias.

2 BROKERS MQTT

2.1 O papel do broker na arquitetura MQTT

O broker é um dos componentes centrais da arquitetura baseada no protocolo MQTT. Ele atua como um intermediário entre os dispositivos que publicam mensagens (publicadores) e os dispositivos que consomem essas mensagens (assinantes), organizando a comunicação por meio de tópicos. Ao contrário de outros modelos ponto a ponto, o broker permite a comunicação assíncrona e desacoplada, na qual não é necessário que o publicador conheça o assinante, nem mantenha uma conexão direta com ele.

Toda a lógica de distribuição das mensagens é responsabilidade do broker. Ele recebe as mensagens dos clientes que publicam dados em um determinado tópico e, em seguida, repassa essas mensagens apenas aos clientes que assinaram aquele tópico. Esse mecanismo garante eficiência e escalabilidade, mesmo em ambientes com milhares de dispositivos conectados.

Além da função de roteamento, os brokers MQTT também podem oferecer funcionalidades adicionais, como autenticação de usuários, controle de acesso a tópicos, persistência de mensagens (para manter mensagens mesmo que os clientes estejam desconectados) e suporte a Qualidade de Serviço (QoS).

A figura a seguir apresenta um exemplo visual da função de um broker MQTT no ecossistema IoT. Nessa representação, os sensores de temperatura, movimento e umidade enviam dados ao broker via protocolo MQTT. O broker, por sua vez, redireciona essas mensagens para plataformas de backend (como bancos de dados NoSQL ou construtores de APIs), e ainda pode interagir com dispositivos móveis e computadores por meio de MQTT ou HTTPS. Esse fluxo ilustra bem o papel central do broker como elo de integração entre dispositivos embarcados e sistemas de análise e controle em nuvem.

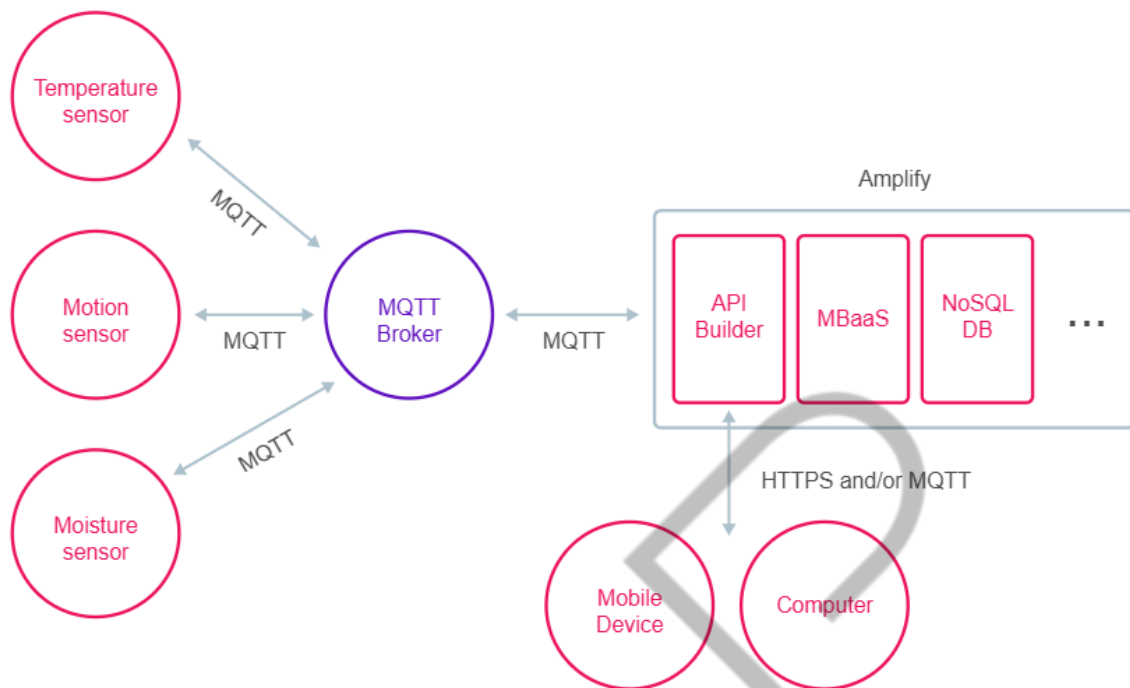


Figura 4 - Representação do papel do broker MQTT no ecossistema IoT, demonstrando a comunicação entre sensores, serviços de backend e dispositivos de acesso

Fonte: ALFACOMP (2025), adaptado por FIAP (2026)

2.2 Utilização de brokers públicos

Para fins educacionais, testes rápidos ou projetos de pequeno porte, é possível utilizar brokers públicos disponíveis gratuitamente na internet. Esses brokers são mantidos por instituições ou empresas e não exigem autenticação, embora ofereçam menos segurança e controle.

Entre os brokers públicos mais utilizados, destacam-se:

- **broker.hivemq.com** – um dos brokers mais acessíveis e amplamente utilizados para testes, sem necessidade de autenticação.
- **test.mosquitto.org** – broker público mantido pela Eclipse Foundation, útil para validar conexões e simular ambientes MQTT.
- **broker.emqx.io** – outro broker gratuito com suporte a MQTT 5.0 e dashboards para visualização.

Ao utilizar brokers públicos, é importante lembrar que as mensagens trafegam em redes compartilhadas e podem ser acessadas por terceiros, caso não haja

criptografia aplicada. Por isso, recomenda-se evitar o envio de dados sensíveis e preferir tópicos únicos e difíceis de adivinhar.

A seguir, o quadro resume as principais características técnicas dos brokers públicos citados, permitindo ao leitor avaliar qual deles é mais adequado ao seu contexto de uso.

Característica	broker.hivemq.com	test.mosquitto.org	broker.emqx.io
Suporte ao MQTT 5.0	Não	Parcial (em testes)	Sim
Autenticação necessária	Não	Não	Não
Conexões simultâneas	Até 100	Até 10.000	Até 1000
Criptografia (TLS)	Sim (opcional)	Sim (portas 8883)	Sim (portas 8883/8884)
Visualização de tópicos	Não	Não	Sim (via dashboard)
Limite de mensagens	Ilimitado (por sessão)	256 KB por mensagem	1 MB por mensagem

Quadro 1 - Comparação entre três brokers públicos MQTT comumente utilizados para fins educacionais e testes em projetos de IoT
Fonte: Elaborado pelo autor (2025)

2.3 Instalação de um broker local com Mosquitto

Em aplicações profissionais, acadêmicas ou quando se deseja maior controle sobre a rede MQTT, a instalação de um broker local se torna fundamental. Um dos brokers mais populares é o Eclipse Mosquitto, um software de código aberto leve e robusto, amplamente utilizado em projetos IoT.

2.3.1 Instalação em diferentes sistemas operacionais

A instalação do Mosquitto varia conforme o sistema operacional:

- **Windows:**
 - 1) Acesse o site oficial: <<https://mosquitto.org/download/>>;
 - 2) Baixe o instalador correspondente à sua versão do Windows;

- 3) Durante a instalação, marque a opção para instalar o serviço como padrão.

- **Linux (Ubuntu/Debian):**

```
sudo apt update
```

```
sudo apt install mosquitto mosquitto-clients
```

- **macOS (Homebrew):**

```
brew install mosquitto
```

```
brew services start mosquitto
```

O serviço inicia automaticamente após a instalação.

2.3.2 Configuração básica do Mosquitto

Após a instalação, o Mosquitto já estará apto a funcionar com uma configuração padrão. No entanto, é possível personalizar o comportamento do broker editando o arquivo de configuração `mosquitto.conf`.

Esse arquivo pode ser encontrado em:

- **Windows:** `C:\Program Files\mosquitto\mosquitto.conf`
- **Linux:** `/etc/mosquitto/mosquitto.conf`
- **macOS:** `/opt/homebrew/etc/mosquitto/mosquitto.conf`

A seguir, apresentamos um trecho funcional de configuração adaptado para testes locais e práticas de laboratório, com explicações embutidas:

```
Ativa o broker na porta padrão MQTT (sem TLS)
listener 1883

# Permite conexões sem autenticação (ideal apenas para
testes locais)
allow_anonymous true
```

O Encontro do IoT com a Nuvem

```
# Habilita a persistência de mensagens (QoS 1 e 2), mesmo
após reinicialização
persistence true
persistence_location /var/lib/mosquitto/

# Define o arquivo de log e os tipos de mensagens a serem
registradas
log_dest file /var/log/mosquitto/mosquitto.log
log_type error
log_type warning
log_type notice
log_type information

# Limites de mensagens em trânsito e em fila por cliente
max_inflight_messages 20 # Mensagens QoS 1 ou 2
aguardando confirmação
max_queued_messages 100 # Máximo de mensagens
armazenadas em fila
max_heartbeat 3600 # Tempo máximo (segundos)
de inatividade do cliente
# Define o QoS máximo aceito (padrão: 2)
max_qos 2

# Limite do tamanho de mensagens aceitas pelo broker (bytes)
message_size_limit 268435455 # Máximo permitido pelo
protocolo MQTT

# Evita que clientes publiquem em tópicos internos do broker
restrict_topic_write $SYS/

# Intervalo de atualização do tópico $SYS
sys_interval 10

# Define comportamento padrão para clientes persistentes
desconectados
persistent_client_expiration 7d # Remove sessões inativas
após 7 dias

# Diretório adicional para incluir configurações (boa
prática modular)
include_dir /etc/mosquitto/conf.d

# --- Segurança (comentado para testes, usar em produção)
---
# password_file /etc/mosquitto/passwd # Arquivo com
usuários e senhas
# acl_file /etc/mosquitto/acl # Arquivo de regras
de acesso a tópicos
# allow_anonymous false # Exige autenticação
```

Código-fonte 1 - Trecho de configuração básica do broker Mosquitto

Fonte: Elaborado pelo autor (2025)

2.3.3 Testes com MQTT Explorer ou comandos MQTT CLI

Com o broker funcionando corretamente, é essencial realizar testes práticos de publicação e assinatura de tópicos. Isso pode ser feito de duas formas principais: utilizando o MQTT Explorer, uma interface gráfica amigável, ou por meio de comandos via terminal, utilizando a ferramenta mosquitto-clients. Ambas as abordagens são importantes e se complementam no processo de aprendizado e depuração.

2.3.3.1 Utilizando o MQTT Explorer

O MQTT Explorer é uma ferramenta gráfica de grande utilidade para compreender visualmente a estrutura e o funcionamento da comunicação MQTT. Ele permite visualizar, publicar e acompanhar mensagens em tempo real, organizadas em tópicos, além de acessar dados técnicos como tamanho das mensagens, QoS e timestamp da última atualização.

Para fins didáticos, este software oferece uma excelente forma de validar a instalação e o funcionamento do broker local, bem como testar a publicação e assinatura de mensagens sem a necessidade de programação.

Instalação do MQTT Explorer:

- 1) Acesse o site oficial: <<https://mqtt-explorer.com>>;
- 2) Faça o download da versão compatível com o seu sistema operacional (Windows, macOS ou Linux);
- 3) Siga as instruções de instalação padrão da sua plataforma.

Após a instalação, abra o MQTT Explorer e siga os passos a seguir para realizar uma conexão com o broker local Mosquitto:

- 1) Clique em "New Connection";
- 2) No campo "Name", insira um nome para identificar sua conexão (exemplo: Broker Local Mosquitto);
- 3) Em "Host", informe localhost ou o IP da máquina onde o Mosquitto está rodando;

- 4) Em "Port", mantenha 1883, que é a porta padrão para conexões MQTT sem TLS;
- 5) Clique em "Connect".

A figura a seguir apresenta a tela inicial do MQTT Explorer, com a conexão bem-sucedida ao broker local (localhost). Nela, é possível observar a árvore de tópicos carregada a partir do broker e a estrutura hierárquica da comunicação.

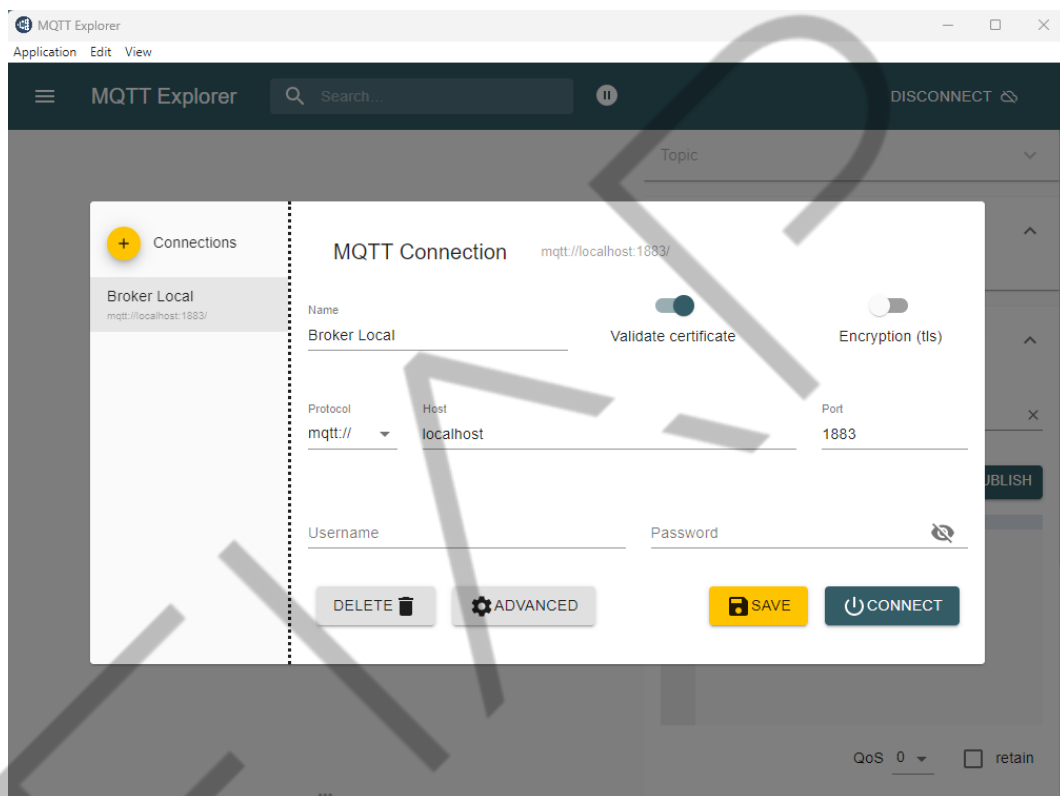


Figura 5 - Tela inicial do MQTT Explorer com árvore de tópicos carregada após conexão com broker local

Fonte: MQTT Explorer (2025)

Publicação de mensagens:

A figura “Publicação manual de mensagens JSON no tópico sensor/distancia utilizando o MQTT Explorer” mostra o processo de envio de mensagens no formato JSON para o tópico "sensor/distancia" utilizando a aba "Publish". Nesta tela, o usuário pode definir o tópico desejado, o tipo de mensagem (JSON, raw ou XML), o QoS, e se deseja que a mensagem seja retida pelo broker. Após preencher os dados, basta clicar em "Publish" para enviar.

O Encontro do IoT com a Nuvem

Como exemplo prático, uma mensagem no formato { "distancia": 35 } pode ser transmitida. Esse envio serve como um teste fundamental para validar se o broker local está aceitando e processando corretamente as publicações de dados.

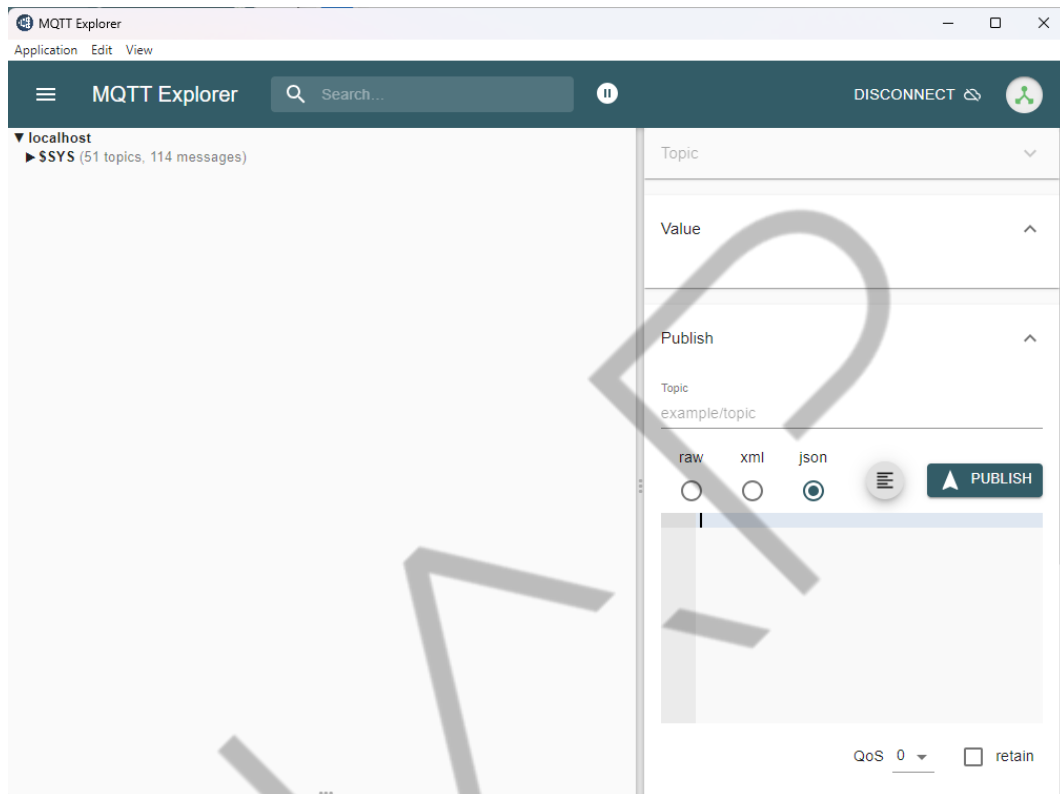


Figura 6 - Publicação manual de mensagens JSON no tópico sensor/distancia utilizando o MQTT Explorer

Fonte: MQTT Explorer (2025)

Visualização e análise dos dados recebidos:

A validação do correto funcionamento do broker local pode ser realizada ao verificar se a mensagem publicada aparece na árvore de tópicos e no painel de visualização do MQTT Explorer. Quando o tópico é selecionado, o painel exibe informações como o conteúdo da última mensagem recebida, o timestamp da recepção, o QoS aplicado e se a mensagem foi marcada como retida. Por exemplo, após a publicação do conteúdo { "distancia": 35 }, espera-se visualizar esse mesmo valor no painel central, confirmando que o broker recebeu e armazenou corretamente a mensagem enviada.

Além disso, a figura a seguir destaca a organização interna do broker, acessada por meio dos tópicos especiais "\$SYS". Esses tópicos fornecem informações detalhadas sobre o status do broker, como número de conexões ativas, quantidade

O Encontro do IoT com a Nuvem

de mensagens publicadas, subscrições e métricas de desempenho, permitindo um monitoramento eficiente da operação.

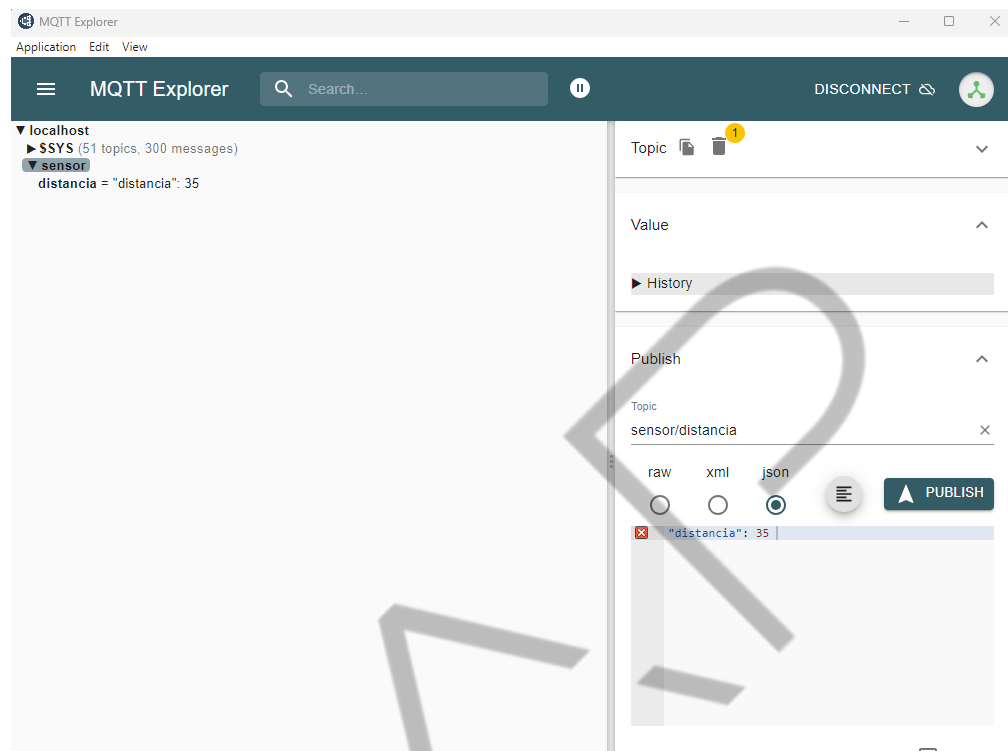


Figura 7 - Tópicos do sistema (\$SYS) mostrando estatísticas do broker, incluindo mensagens, clientes e conexões ativas

Fonte: MQTT Explorer (2025)

Essas visualizações permitem validar se os dados estão sendo corretamente recebidos, se os tópicos estão estruturados conforme esperado e se o broker está operando de forma estável. O MQTT Explorer se torna, assim, uma ferramenta essencial durante o processo de desenvolvimento e teste de sistemas baseados em MQTT.

2.3.3.2 Utilizando o Mosquitto CLI

A linha de comando do Mosquitto é uma alternativa poderosa e leve para realizar testes MQTT, sem a necessidade de interfaces gráficas. Ela é ideal para validar rapidamente se o broker está funcionando, bem como para automatizar tarefas de publicação e subscrição de mensagens.

Execução no terminal:

Os comandos `mosquitto_pub` e `mosquitto_sub` devem ser executados diretamente no terminal do sistema operacional. Isso pode ser feito através do Prompt de Comando (CMD) ou PowerShell no Windows, ou ainda no Terminal no Linux ou macOS.

Subscrição com `mosquitto_sub`:

O primeiro passo do teste é abrir um terminal e executar o comando para ficar escutando as mensagens recebidas no tópico desejado. Use o seguinte comando:

```
mosquitto_sub -h localhost -p 1883 -t sensor/distancia
```

Comando de prompt 1 - Comando para iniciar a escuta no tópico `sensor/distancia` através do terminal, utilizando o broker MQTT local na porta 1883

Fonte: Elaborado pelo autor (2025)

Esse comando irá manter o terminal aberto e escutando o tópico `sensor/distancia`, aguardando mensagens. A próxima figura apresenta esse terminal aguardando a chegada de dados.

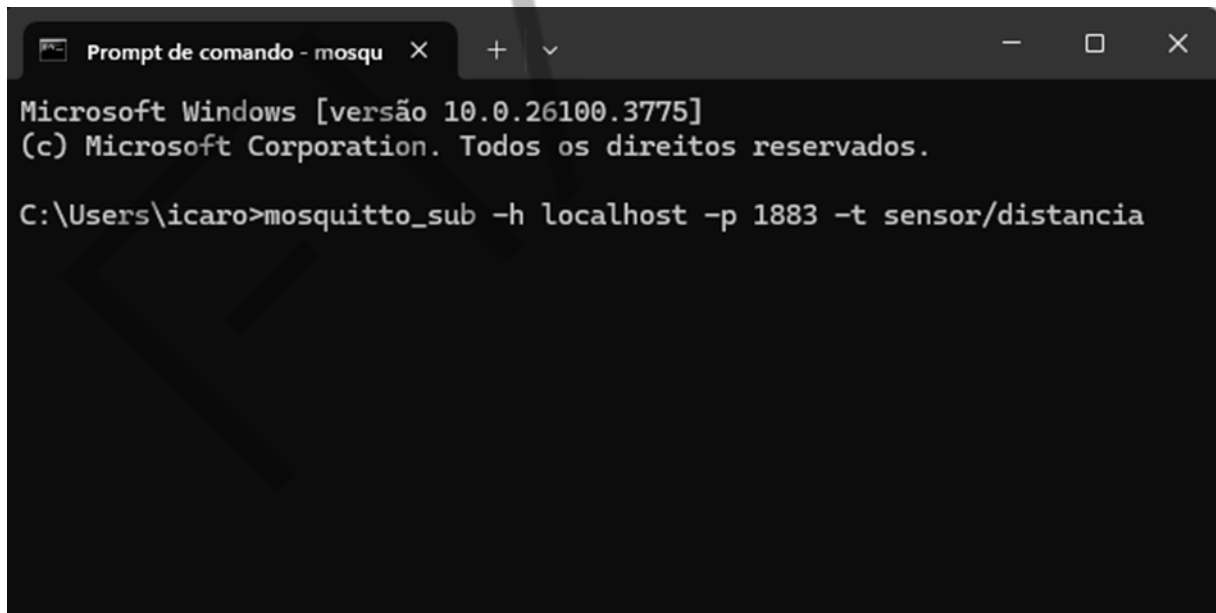


Figura 8 - Execução do comando `mosquitto_sub` aguardando mensagens no tópico `sensor/distancia`

Fonte: Elaborado pelo autor (2025)

Publicação de mensagens com `mosquitto_pub`:

Em outro terminal, execute o comando para publicar a mensagem. No exemplo a seguir, será enviada uma mensagem com o valor de distância igual a 35:

```
mosquitto_pub -h localhost -p 1883 -t sensor/distancia -m  
"{\"distancia\": 35}"
```

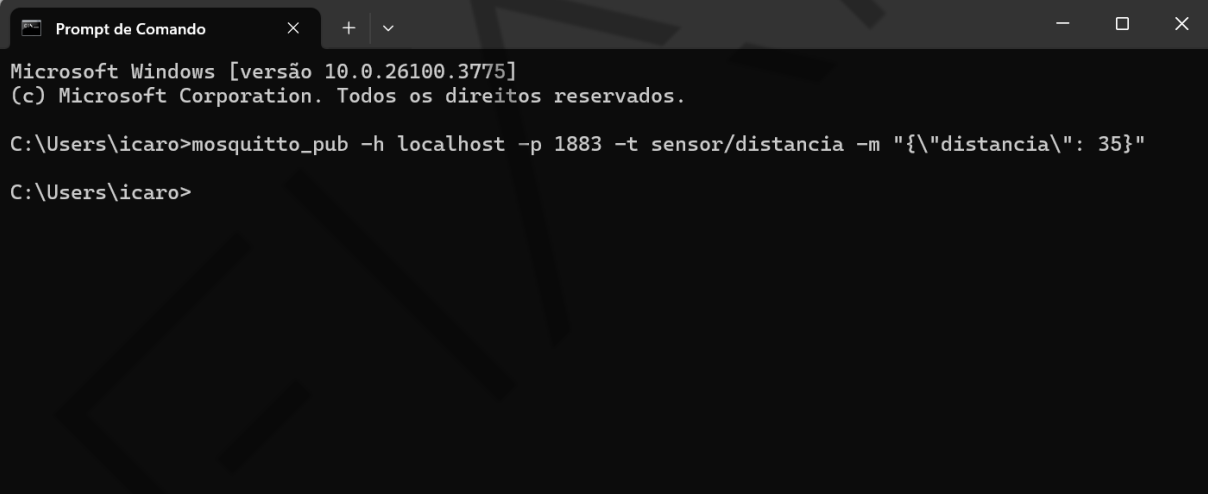
Comando de prompt 2 - Publicação manual de uma mensagem JSON com valor de distância igual a 35, simulando um dispositivo publicador em um tópico MQTT

Fonte: Elaborado pelo autor (2025)

Neste comando:

- `-h localhost` define que o broker está local.
- `-p 1883` especifica a porta padrão do MQTT.
- `-t sensor/distancia` indica o tópico da mensagem.
- `-m` define a mensagem enviada.

A figura a seguir mostra a execução bem-sucedida do comando `mosquitto_pub`, sem apresentar erros.



```
Prompt de Comando
Microsoft Windows [versão 10.0.26100.3775]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\icaro>mosquitto_pub -h localhost -p 1883 -t sensor/distancia -m "{\"distancia\": 35}"

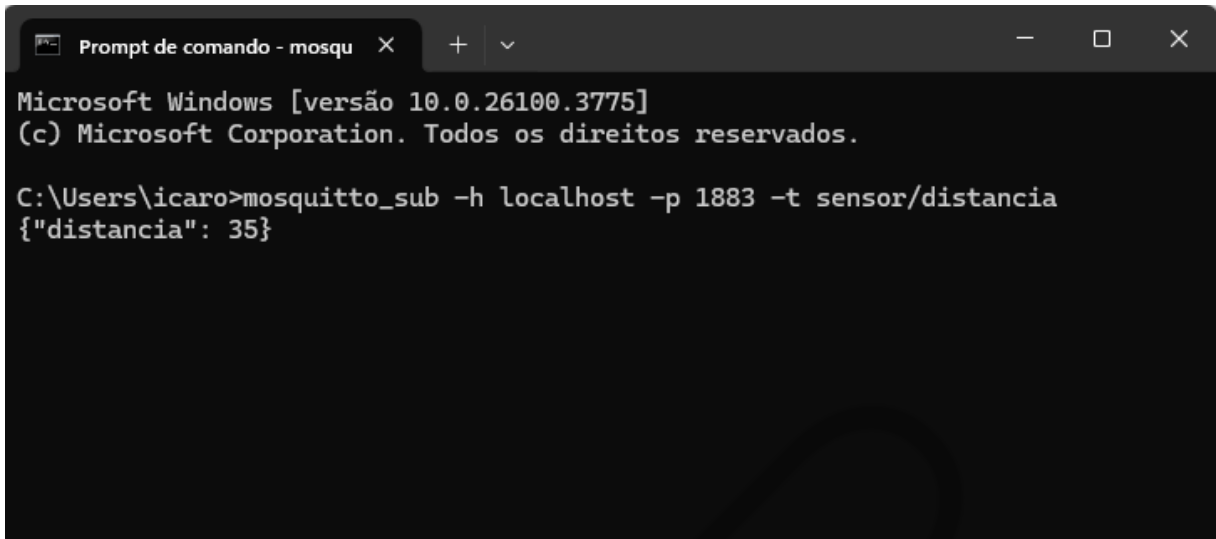
C:\Users\icaro>
```

Figura 9 - Comando `mosquitto_pub` enviado com sucesso para o tópico `sensor/distancia`

Fonte: Elaborado pelo autor (2025)

Visualização e análise dos dados recebidos:

Se o broker estiver funcionando corretamente, o terminal que executa o `mosquitto_sub` deverá exibir automaticamente a mensagem publicada, como ilustra a figura a seguir.

A screenshot of a Windows Command Prompt window titled "Prompt de comando - mosqu". The window shows the output of the Mosquitto CLI. It displays the Windows version (10.0.26100.3775) and copyright information. Then, the command `mosquitto_sub -h localhost -p 1883 -t sensor/distancia` is entered. The output shows a received JSON message: `{"distancia": 35}`.

```
Prompt de comando - mosqu X + v
Microsoft Windows [versão 10.0.26100.3775]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\icaro>mosquitto_sub -h localhost -p 1883 -t sensor/distancia
{"distancia": 35}
```

Figura 10 - Terminal exibindo a mensagem recebida, validando o funcionamento do broker Mosquitto local

Fonte: Elaborado pelo autor (2025)

Essa abordagem simples com a CLI do Mosquitto permite validar o fluxo completo de publicação e recepção de mensagens, sendo ideal para testes rápidos, automações e diagnósticos.

3 EXPERIMENTOS COM MQTT E ESP32

Esta seção tem como objetivo apresentar experiências práticas com a comunicação MQTT utilizando ESP32 e sensores. Os experimentos simulam uma aplicação real de semáforo inteligente, que detecta a presença de pedestres por meio de sensores ultrassônicos e realiza a comutação do semáforo com base nas mensagens transmitidas pelo protocolo MQTT.

A figura a seguir apresenta o diagrama geral do experimento, permitindo visualizar a comunicação entre os dispositivos, a lógica de publicação e assinatura via MQTT, e o papel de cada elemento no sistema.

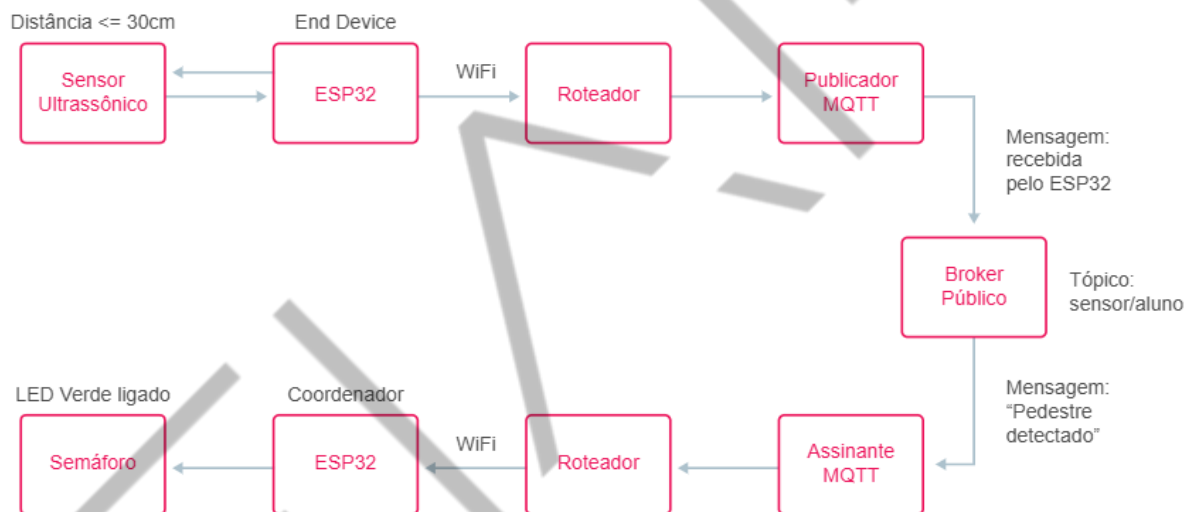


Figura 11 - Diagrama geral do experimento de semáforo inteligente com ESP32 e MQTT
Fonte: Elaborado pelo autor (2025)

3.1 Fundamentos do circuito: sensor ultrassônico e LEDs

Antes de explorarmos o protocolo MQTT em nossos experimentos, é fundamental compreender o funcionamento dos componentes eletrônicos utilizados no projeto, especialmente o sensor ultrassônico HC-SR04 e os LEDs que compõem o semáforo inteligente.

Funcionamento do sensor ultrassônico HC-SR04:

O sensor HC-SR04 é capaz de medir a distância entre o sensor e um obstáculo, com base na emissão e recepção de pulsos ultrassônicos. O processo funciona da seguinte maneira:

- O pino TRIG (trigger) envia um pulso de 10 microssegundos.
- O pino ECHO (echo) detecta o tempo que o pulso levou para retornar após refletir em um objeto.
- Com base nesse tempo, a distância é calculada pela fórmula:

$$Distância\ (cm) = \frac{tempo\ do\ ECHO \times 0.034}{2}$$

A velocidade do som no ar é de aproximadamente 0,034 cm/μs. Dividimos por dois porque o pulso percorre o caminho de ida e volta.

A figura a seguir apresenta o diagrama de ligação entre o sensor ultrassônico HC-SR04 e o ESP32.

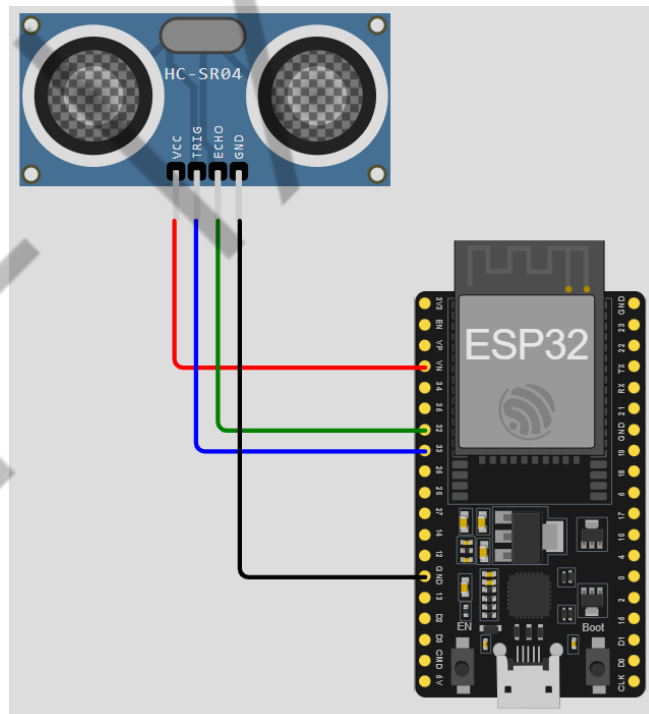


Figura 12 - Diagrama de ligação entre o sensor ultrassônico HC-SR04 e o ESP32
Fonte: Wokwi (2025)

Acionamento de LEDs e cálculo de corrente:

O projeto utiliza três LEDs (verde, amarelo e vermelho) para simular o funcionamento de um semáforo. Cada LED é conectado a um pino digital do ESP32 por meio de um resistor, responsável por limitar a corrente. Considerando uma tensão de 3,3 V do ESP32 e uma queda de tensão típica de 1,8 V em um LED (conforme especificações do fabricante), e visando uma corrente de operação segura de 20 mA, o valor ideal do resistor pode ser calculado pela Lei de Ohm:

$$R = \frac{3.3V - 1.8V}{0.02A} = 75\Omega$$

Assim, o valor recomendado para o resistor é de 75 ohms, o que garante o brilho adequado do LED e protege tanto o componente quanto o microcontrolador. Caso esse valor não esteja disponível, pode-se utilizar um resistor de valor próximo, como 82 ohms. Essa escolha é baseada nas características de operação típicas de LEDs, como mostrado na imagem de especificação técnica consultada.

A figura a seguir apresenta o diagrama de ligação dos LEDs ao ESP32 simulando um semáforo.

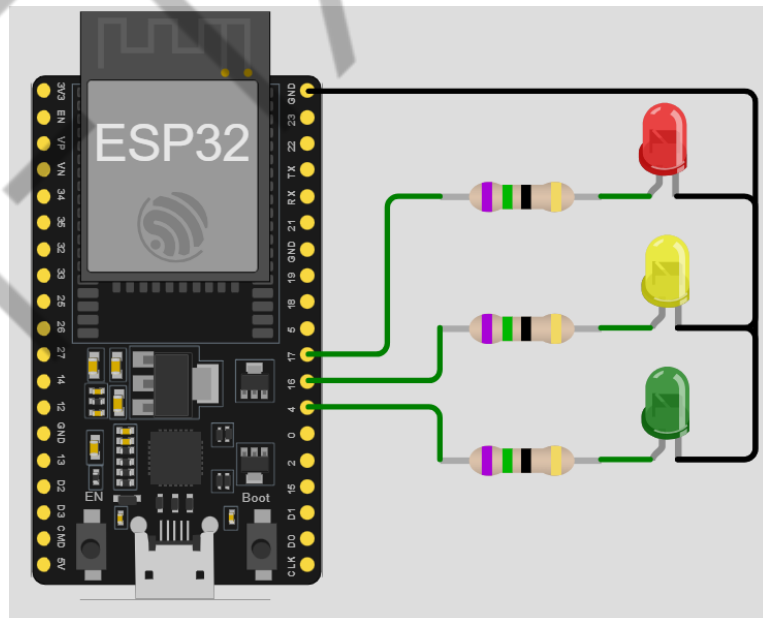


Figura 13 - Diagrama de ligação dos LEDs ao ESP32 simulando um semáforo
Fonte: Wokwi (2025)

A escolha dos pinos foi feita com base na facilidade de uso e acessibilidade física no modelo ESP32 DevKit. Utilizamos os seguintes pinos:

- **TRIG do sensor: GPIO 33** — pino digital de uso geral, não reservado internamente.
- **ECHO do sensor: GPIO 32** — também de uso geral e ideal para entrada digital.
- **LED Verde: GPIO 4.**
- **LED Amarelo: GPIO 17.**
- **LED Vermelho: GPIO 16.**

Todos os pinos escolhidos estão livres de restrições específicas (como uso de ADC, SPI ou comunicação interna), o que os torna ideais para controle de LEDs e leitura digital.

Código simples de acionamento do sensor:

A seguir é apresentado um exemplo de código básico apenas para medir e exibir a distância com o sensor ultrassônico.

```
#define TRIG_PIN 33
#define ECHO_PIN 32

void setup() {
  Serial.begin(115200); // Inicializa a comunicação serial
  pinMode(TRIG_PIN, OUTPUT); // Define o pino TRIG como
saída
  pinMode(ECHO_PIN, INPUT); // Define o pino ECHO como
entrada
}

void loop() {
  // Envia um pulso de 10 microssegundos no pino TRIG
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Mede o tempo do pulso no pino ECHO
  long duracao = pulseIn(ECHO_PIN, HIGH);
```

```
// Calcula a distância com base no tempo
long distancia = duracao * 0.034 / 2;

// Exibe a distância no monitor serial
Serial.print("Distância: ");
Serial.print(distancia);
Serial.println(" cm");
delay(1000); // Aguarda 1 segundo antes da próxima medição
}
```

Código-fonte 2 - Código de leitura do sensor ultrassônico HC-SR04 com ESP32
Fonte: Elaborado pelo autor (2025)

Esse código permite verificar, via Monitor Serial, o funcionamento correto do sensor antes de integrar o protocolo MQTT e a lógica do semáforo inteligente.

3.2 Enviando dados com sensor ultrassônico (publicador)

Nesta etapa, utilizamos o ESP32 com um sensor ultrassônico para detectar a presença de um objeto a curta distância (menor que 30 cm). Quando isso ocorre, o dispositivo envia uma mensagem via protocolo MQTT para um broker público, simulando o comportamento de um publicador.

Para isso, foi utilizado o broker gratuito broker.emqx.io, que permite conexões na porta 1883 sem autenticação. O tópico utilizado na publicação foi sensor/aluno. O payload foi estruturado em formato JSON, contendo o nome do dispositivo e o valor da distância detectada:

```
{
  "dispositivo": "ESP32_SENSOR_ULTRASSONICO",
  "distancia": 6
}
```

Código-fonte 3 - Exemplo de mensagem JSON formatada para envio pelo dispositivo ESP32 com sensor ultrassônico, representando o payload publicado via protocolo MQTT
Fonte: Elaborado pelo autor (2025)

O código utilizado neste experimento realiza as seguintes operações:

- Medição da distância via sensor ultrassônico HC-SR04;
- Conexão à rede Wi-Fi e ao broker MQTT público;
- Publicação periódica de mensagens no tópico sensor/aluno com os dados em formato JSON;

O Encontro do IoT com a Nuvem

- A mensagem publicada contém o nome do dispositivo e a distância detectada.

A seguir é apresentado o código completo utilizado para realizar a lógica de conexão Wi-Fi, conexão MQTT, medição de distância com o sensor ultrassônico e envio da mensagem JSON:

```
#include <WiFi.h>           // Biblioteca para conexão Wi-Fi
no ESP32
#include <PubSubClient.h> // Biblioteca para comunicação
MQTT

// Configurações da rede Wi-Fi
#define WIFI_SSID "FIAP-IOT"
#define WIFI_PASSWORD "F!@p25.IOT"

// Configurações do broker MQTT público
#define MQTT_SERVER "broker.emqx.io"
#define MQTT_PORT 1883
#define MQTT_TOPIC "sensor/aluno"

// Instanciando cliente Wi-Fi e cliente MQTT
WiFiClient espClient;
PubSubClient client(espClient);

// Pinos do sensor ultrassônico
#define TRIG_PIN 33
#define ECHO_PIN 32

// Identificador do dispositivo
#define MEU_NOME "ESP32_SENSOR_ULTRASSONICO"

// Função de configuração inicial do ESP32
void setup() {
    Serial.begin(115200); // Inicia comunicação serial para
    depuração
    pinMode(TRIG_PIN, OUTPUT); // Define TRIG como saída
    pinMode(ECHO_PIN, INPUT);  // Define ECHO como entrada
    conectarWiFi(); // Conecta à rede Wi-Fi
    // Define servidor MQTT
    client.setServer(MQTT_SERVER, MQTT_PORT);
}

// Função para conectar ao Wi-Fi
void conectarWiFi() {
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
}
```

O Encontro do IoT com a Nuvem

```
    }
    Serial.println(" Conectado ao WiFi!");
}
// Função para conectar ao broker MQTT
void conectarMQTT() {
    while (!client.connected()) {
        Serial.print("Conectando ao broker MQTT...");
        if (client.connect(MEU_NOME)) {
            Serial.println(" Conectado ao MQTT!");
        } else {
            Serial.print(" Falha, rc=");
            Serial.print(client.state());
            Serial.println(" Tentando novamente em 5
segundos...");
            delay(5000);
        }
    }
}

// Função que mede a distância com o sensor ultrassônico
long medirDistancia() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH); // Pulso de 10 µs para
iniciar medição
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duracao = pulseIn(ECHO_PIN, HIGH); // Tempo em
microssegundos
    return duracao * 0.034 / 2; // Conversão para centímetros
}

// Loop principal do programa
void loop() {
    if (!client.connected()) {
        conectarMQTT(); // Garante conexão ao broker
    }
    client.loop(); // Mantém a conexão MQTT ativa

    long distancia = medirDistancia();
    Serial.println("Distância medida: " + String(distancia)
+ " cm");

    if (distancia < 30) {
        Serial.println("Pessoa detectada! Enviando dados...");

        // Monta mensagem JSON
        String payload = "{ \"dispositivo\": \"" +
String(MEU_NOME) + "\", \"distancia\": " + String(distancia) +
" }";
```

O Encontro do IoT com a Nuvem

```
client.publish(MQTT_TOPIC, payload.c_str()); // Envia a mensagem
delay(2000); // Aguarda antes de nova detecção
}

delay(500); // Intervalo entre medições
}
```

Código-fonte 4 - Publicador MQTT com sensor ultrassônico via ESP32
Fonte: Elaborado pelo autor (2025)

A primeira etapa consiste em configurar a conexão no MQTT Explorer. Como foi utilizado um broker público (broker.emqx.io), basta informar o endereço e a porta 1883, sem necessidade de autenticação. A figura “Configuração da conexão com o broker público no MQTT Explorer” ilustra como essa configuração é realizada.

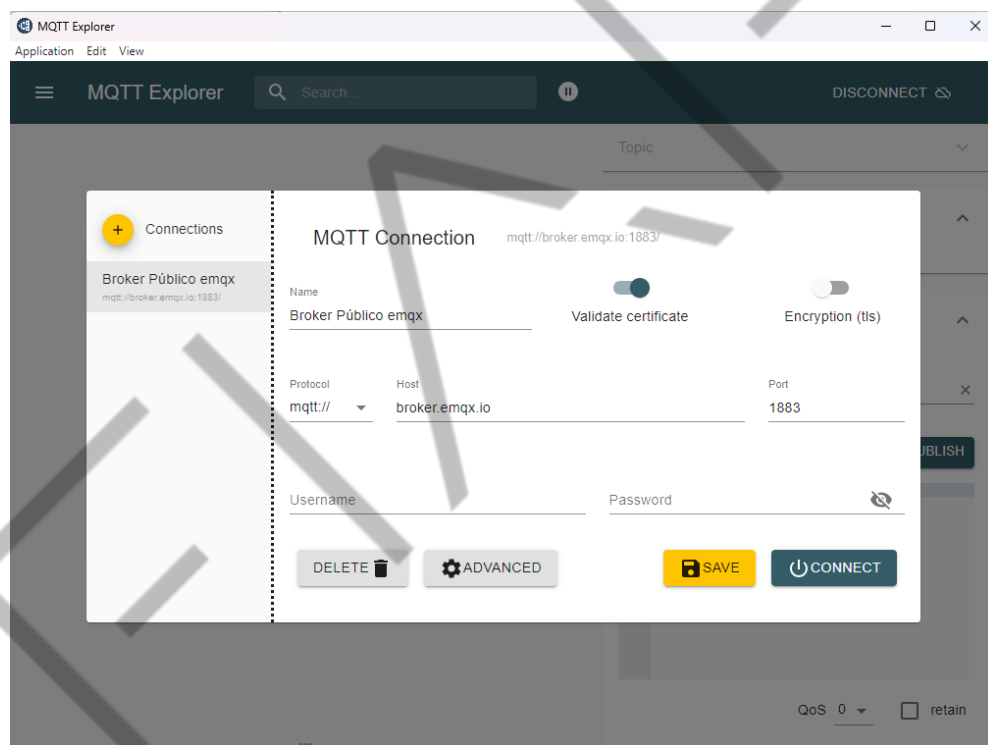


Figura 14 - Configuração da conexão com o broker público no MQTT Explorer
Fonte: MQTT Explorer (2025)

3.2.1 Assinatura do Tópico

Após o estabelecimento da conexão, o tópico sensor/aluno é adicionado ao MQTT Explorer para que o cliente possa escutar as mensagens enviadas pelo ESP32. Para isso, foi necessário clicar no botão ADVANCED, escrever o tópico e, em seguida, clicar no botão +ADD. Essa etapa é fundamental para validar a comunicação.

Observe a figura:

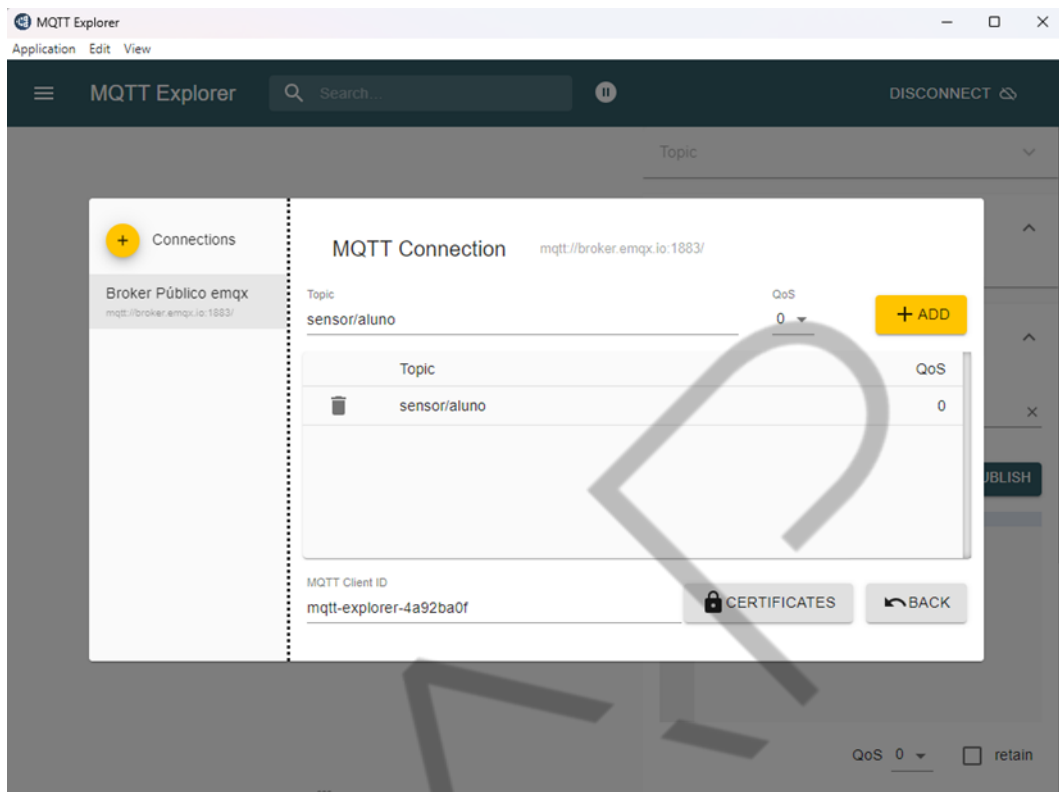


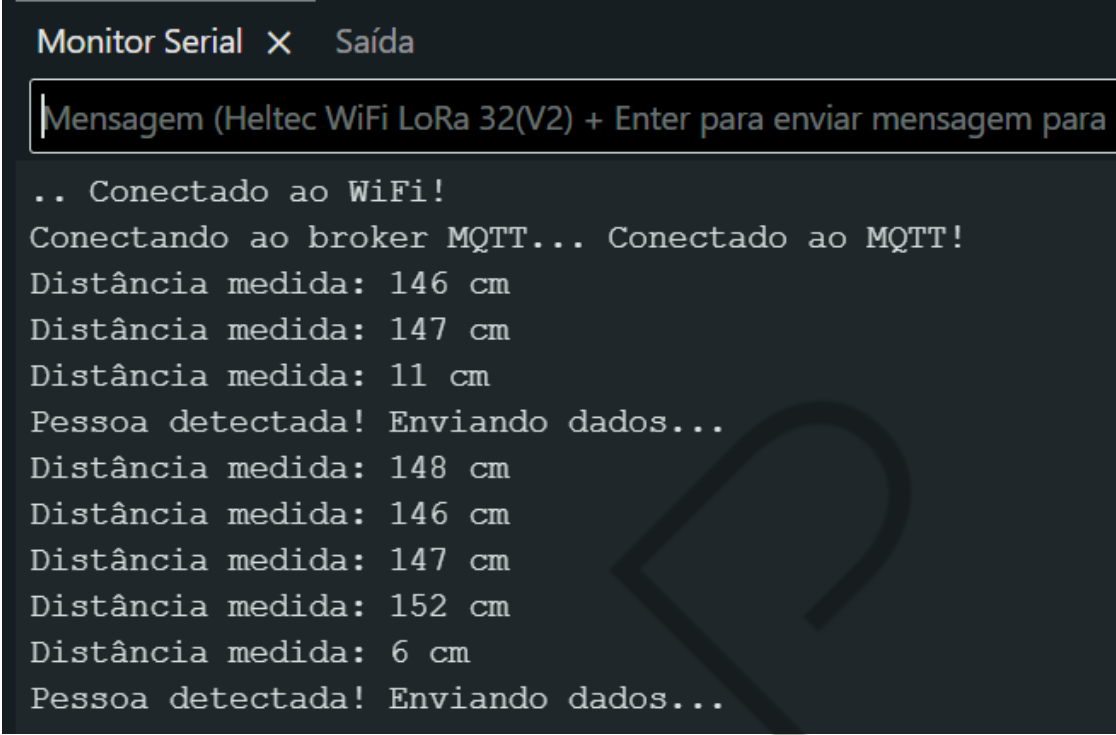
Figura 15 - Assinatura do tópico sensor/aluno no MQTT Explorer
Fonte: MQTT Explorer (2025)

3.2.2 Envio e Recepção de Mensagens

Com o ESP32 conectado à rede Wi-Fi e ao broker, o código embarcado no microcontrolador envia mensagens em formato JSON contendo o nome do dispositivo e a distância medida, sempre que o sensor detectar um objeto a menos de 30 cm.

Durante os testes, duas mensagens foram transmitidas com sucesso, registrando as distâncias de 11 cm e 6 cm.

Observe na figura a seguir que esses valores foram exibidos no monitor serial do Arduino IDE. Essa visualização permite validar que o sensor está funcionando corretamente e que a lógica de envio da mensagem está operacional.

The image shows a screenshot of the 'Monitor Serial' window in the Arduino IDE. The window title is 'Monitor Serial' with a close button (X) and a label 'Saída'. The input field at the top contains the text 'Mensagem (Heltec WiFi LoRa 32(V2) + Enter para enviar mensagem para'. The output area displays the following text: '.. Conectado ao WiFi!', 'Conectando ao broker MQTT... Conectado ao MQTT!', 'Distância medida: 146 cm', 'Distância medida: 147 cm', 'Distância medida: 11 cm', 'Pessoa detectada! Enviando dados...', 'Distância medida: 148 cm', 'Distância medida: 146 cm', 'Distância medida: 147 cm', 'Distância medida: 152 cm', 'Distância medida: 6 cm', and 'Pessoa detectada! Enviando dados...'. A large, faint watermark 'E' is visible in the background of the serial monitor area.

```
Monitor Serial X Saída
Mensagem (Heltec WiFi LoRa 32(V2) + Enter para enviar mensagem para

.. Conectado ao WiFi!
Conectando ao broker MQTT... Conectado ao MQTT!
Distância medida: 146 cm
Distância medida: 147 cm
Distância medida: 11 cm
Pessoa detectada! Enviando dados...
Distância medida: 148 cm
Distância medida: 146 cm
Distância medida: 147 cm
Distância medida: 152 cm
Distância medida: 6 cm
Pessoa detectada! Enviando dados...
```

Figura 16 - Mensagens recebidas no monitor serial do ESP32 indicando distâncias inferiores a 30 cm
Fonte: Arduino IDE (2025)

Em um segundo momento, essas mesmas mensagens foram visualizadas no MQTT Explorer, comprovando que o envio realizado pelo ESP32 foi corretamente recebido pelo cliente assinante.

Observe:

O Encontro do IoT com a Nuvem

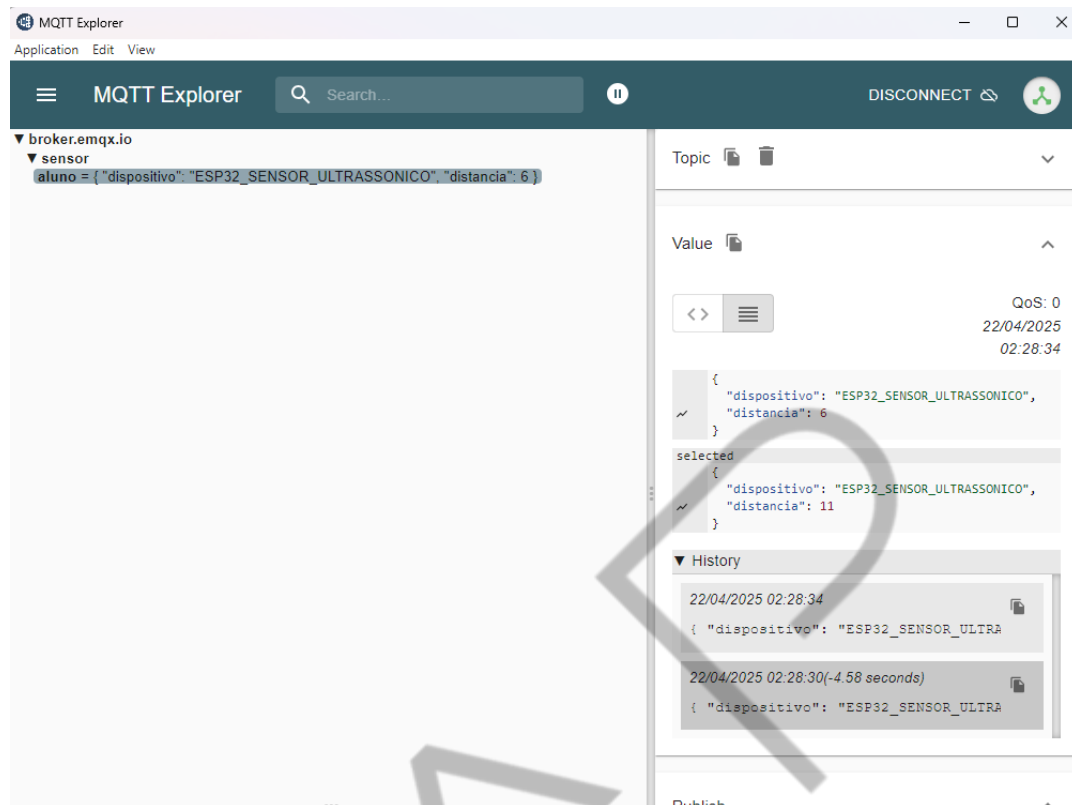


Figura 17 - Mensagens recebidas no tópico sensor/aluno no MQTT Explorer
Fonte: MQTT Explorer (2025)

O ESP32 com sensor ultrassônico funciona como publicador na arquitetura MQTT, enquanto o MQTT Explorer atua como cliente assinante, recebendo em tempo real os dados publicados no tópico sensor/aluno.

3.3 Recebendo dados e atuando com semáforo inteligente (assinante)

Neste experimento, utilizamos um segundo ESP32 para atuar como assinante do protocolo MQTT, reagindo a mensagens publicadas por outro dispositivo com o sensor ultrassônico. A ideia é controlar um semáforo simples (composto por LEDs vermelho, amarelo e verde) com base na detecção de presença, ilustrando a comunicação entre dispositivos IoT.

A seguir, o código embarcado no ESP32 coordenador é apresentado e comentado linha por linha, para facilitar a compreensão do funcionamento.

O Encontro do IoT com a Nuvem

```
#include <WiFi.h> // Biblioteca para conexão Wi-Fi
no ESP32
#include <PubSubClient.h> // Biblioteca para comunicação
MQTT

// Configurações da rede Wi-Fi
const char* ssid = "FIAP-IOT";
const char* password = "F!@p25.IOT";

// Configurações do broker MQTT (público)
const char* mqtt_server = "broker.emqx.io";
const int mqtt_port = 1883;
const char* mqtt_topic = "sensor/aluno"; // Tópico que será
assinado

WiFiClient espClient; // Cria cliente Wi-Fi
PubSubClient client(espClient); // Cria cliente MQTT
usando o cliente Wi-Fi

// Definição dos pinos dos LEDs do semáforo
#define LED_VERMELHO 17
#define LED_AMARELO 16
#define LED_VERDE 4

// Número de pessoas diferentes para acionar o semáforo
#define PESSOAS_NECESSARIAS 1
String nomesRecebidos[PESSOAS_NECESSARIAS]; // Lista de
nomes recebidos
int totalNomes = 0; // Contador de nomes distintos

void setup() {
    Serial.begin(115200); // Inicializa a comunicação serial

    // Define os pinos dos LEDs como saída
    pinMode(LED_VERMELHO, OUTPUT);
    pinMode(LED_AMARELO, OUTPUT);
    pinMode(LED_VERDE, OUTPUT);

    // Inicializa o semáforo com luz verde ligada
    digitalWrite(LED_VERDE, HIGH);
    digitalWrite(LED_AMARELO, LOW);
    digitalWrite(LED_VERMELHO, LOW);

    conectarWiFi(); // Conecta ao Wi-Fi
    client.setServer(mqtt_server, mqtt_port); // Configura o
broker MQTT
    client.setCallback(callback); // Define a função de
callback para recebimento
}
```

O Encontro do IoT com a Nuvem

```
// Conexão à rede Wi-Fi
void conectarWiFi() {
    Serial.print("Conectando ao WiFi...");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println(" Conectado ao WiFi!");
}

// Conexão ao broker MQTT e assinatura do tópico
void conectarMQTT() {
    while (!client.connected()) {
        Serial.print("Conectando ao broker MQTT...");
        if (client.connect("ESP32_Coordenador")) {
            Serial.println(" Conectado ao MQTT!");
            client.subscribe(mqtt_topic); // Se inscreve no
tópico
        } else {
            Serial.print(" Falha, rc=");
            Serial.print(client.state());
            Serial.println(" Tentando novamente em 5
segundos...");
            delay(5000);
        }
    }
}

// Função chamada ao receber uma nova mensagem
void callback(char* topic, byte* payload, unsigned int
length) {
    String mensagemRecebida = "";
    for (unsigned int i = 0; i < length; i++) {
        mensagemRecebida += (char)payload[i]; // Concatena os
bytes recebidos
    }

    Serial.println("Mensagem recebida: " + mensagemRecebida);

    // Extração dos campos "dispositivo" e "distancia" do
JSON recebido
    int nomeInicio =
mensagemRecebida.indexOf("\"dispositivo\":\"); + 14;
    int nomeFim = mensagemRecebida.indexOf("\"",",",
nomeInicio);
    int distanciaInicio =
mensagemRecebida.indexOf("\"distancia\":") + 11;
```

O Encontro do IoT com a Nuvem

```
        int    distanciaFim    =    mensagemRecebida.indexOf("}",
distanciaInicio);

        if (nomeInicio > 0 && nomeFim > nomeInicio &&
distanciaInicio > 0 && distanciaFim > distanciaInicio) {
            String                nomeRecebido                =
mensagemRecebida.substring(nomeInicio, nomeFim);
            String                distanciaRecebida            =
mensagemRecebida.substring(distanciaInicio, distanciaFim);

            Serial.println("Pessoa detectada: " + nomeRecebido + "
| Distância: " + distanciaRecebida + " cm");

            // Verifica se o nome já foi registrado
            bool nomeExiste = false;
            for (int i = 0; i < totalNomes; i++) {
                if (nomesRecebidos[i] == nomeRecebido) {
                    nomeExiste = true;
                    break;
                }
            }

            // Se for um novo nome, adiciona à lista
            if (!nomeExiste) {
                nomesRecebidos[totalNomes] = nomeRecebido;
                totalNomes++;
            }

            // Verifica se atingiu o número necessário de pessoas
            if (totalNomes >= PESSOAS_NECESSARIAS) {
                Serial.println("Quantidade de pessoas atingida!
Mudando semáforo...");

                // Fase amarela
                digitalWrite(LED_VERMELHO, LOW);
                digitalWrite(LED_AMARELO, HIGH);
                digitalWrite(LED_VERDE, LOW);
                delay(2000);

                // Fase vermelha
                digitalWrite(LED_VERMELHO, HIGH);
                digitalWrite(LED_AMARELO, LOW);
                digitalWrite(LED_VERDE, LOW);
                delay(5000);

                // Retorna à fase verde
                digitalWrite(LED_VERMELHO, LOW);
                digitalWrite(LED_AMARELO, LOW);
                digitalWrite(LED_VERDE, HIGH);
            }
        }
    }
}
```

```
        totalNomes = 0; // Reinicia o contador
    }
}
}

// Loop principal que mantém a conexão com o broker ativa
void loop() {
    if (!client.connected()) {
        conectarMQTT(); // Tenta reconectar ao broker caso
desconectado
    }
    client.loop(); // Mantém a comunicação MQTT funcionando
}
```

Código-fonte 5 - Assinante MQTT com ESP32 e controle de semáforo via LED

Fonte: Elaborado pelo autor (2025)

Este código exemplifica uma aplicação prática de IoT, demonstrando a interação entre um sensor (publicador) e um atuador (assinante) através do protocolo MQTT. A lógica implementada considera a presença de pessoas com base na distância medida por outro ESP32 com sensor ultrassônico. Quando a quantidade mínima de detecções únicas é atingida, o semáforo realiza a troca de estado de forma sequencial: verde → amarelo → vermelho → verde novamente.

4 APLICAÇÕES REAIS E REFLEXÕES

O experimento apresentado neste módulo vai muito além de uma simples demonstração de funcionamento do protocolo MQTT. Ele reflete um cenário real cada vez mais comum na Indústria 4.0, em ambientes urbanos inteligentes e na automação de processos industriais ou residenciais. O uso de sensores para coleta de dados e atuadores para tomada de decisão descentralizada é a base da IoT.

No caso do semáforo inteligente, por exemplo, podemos imaginar diversas aplicações práticas:

- Controle de acesso a ambientes baseado na presença de pessoas ou objetos detectados por sensores.
- Sinalização inteligente em corredores ou entradas de locais restritos, como hospitais ou centros de logística.
- Gerenciamento de fluxo de pessoas, por exemplo em eventos, utilizando contagem de dispositivos conectados ou sensores de distância.

Outro aspecto importante é a escalabilidade e modularidade do protocolo MQTT. A estrutura publish/subscribe permite adicionar novos sensores e atuadores facilmente, sem a necessidade de alteração no código de todos os dispositivos. Basta que estejam escutando ou publicando nos tópicos corretos. Isso garante robustez, flexibilidade e economia de recursos computacionais.

Também vale refletir sobre os desafios que surgem ao se colocar esse tipo de sistema em operação real:

- **Segurança e autenticação:** o uso de brokers públicos não é recomendado para aplicações sensíveis. É essencial aplicar criptografia (TLS) e autenticação adequada para proteger os dados transmitidos.
- **Latência e confiabilidade:** para aplicações críticas, como saúde ou indústria, a escolha do QoS adequado e a infraestrutura da rede devem garantir entregas confiáveis e em tempo hábil. Nesses casos, uma solução viável é o uso de um broker MQTT local, instalado na própria rede da organização, o que reduz significativamente a latência e elimina dependências externas.

- **Manutenção e atualizações remotas:** dispositivos IoT em campo precisam de estratégias para diagnóstico, monitoramento e atualizações.

Em resumo, o experimento do semáforo com ESP32, sensor ultrassônico e protocolo MQTT serve como um microcosmo de sistemas reais que estão transformando o cotidiano das cidades e dos processos produtivos. Ele demonstra como a comunicação em tempo real entre dispositivos pode ser aplicada para tomada de decisão automatizada, descentralizada e eficiente, reforçando os pilares de um mundo cada vez mais conectado.

REFERÊNCIAS

ALFACOMP. MQTT: conheça o protocolo padrão do IoT. **AlfaComp**, 2022. Disponível em: <<https://alfacomp.net/2022/03/24/mqtt-conheca-o-protocolo-padrao-do-iot/>>. Acesso em: 20 abr. 2025.

ARDUINO. Arduino IDE - Ambiente de desenvolvimento integrado. **Arduino.cc**, 2025. Disponível em: <<https://www.arduino.cc/en/software>>. Acesso em: 21 abr. 2025.

FARIA, Júnior; Paulo Henrique. Redes de Comunicação em IoT – Protocolo ZigBee. **Universidade Federal do Espírito Santo (UFES)**, [s.d.]. Disponível em: <https://www.inf.ufes.br/~phfaria/disciplinas/iot/material/ZigBee_RedesIoT.pdf>. Acesso em: 23 abr. 2025.

NORDQUIST, Thomas. Visualizador e depurador de tópicos MQTT. Versão 0.4.0. **MQTT Explorer**, [s.d.]. Disponível em: <<https://mqtt-explorer.com/>>. Acesso em: 23 abr. 2025.

TEDESCHI, Pietro; SCIANCALEPORE, Savio. **Edge and Fog Computing in Critical Infrastructures: Analysis, Security Threats, and Research Challenges**. Doha, Qatar: Hamad Bin Khalifa University, 2019. (Figura adaptada). Acesso em: 23 abr. 2025.

WOKWI. Simulador online de circuitos e microcontroladores. **Wokwi**, 2025. Disponível em: <<https://wokwi.com/>>. Acesso em: 19 abr. 2025.